RIYADVI TECHNOLOGIES

SOFTWARE PROJECT PLANNING GUIDE

From idea to impact. Plan with purpose.

A practical framework to define the problem, align stakeholders, design a solution, build in iterations, and prepare for launch.

DISCOVER • DEFINE • DESIGN • BUILD • LAUNCH

Use this guide as a working document. Adapt the sequence, deliverables, and timing to your project scope, team size, dependencies, and feedback cycle.

How to use this guide

Start with the business outcome, not a list of features. Work through each phase, record decisions, and agree on what “done” means before moving forward. Smaller projects may combine phases; complex projects may repeat them.

Planning principle: each phase should produce an understandable output that the team and stakeholders can review.

01 — Discover the problem

Suggested timing: Week 1

Clarify the business problem, intended users, current workflow, and why a change is needed.

- Write a one-paragraph problem statement.
- Identify stakeholders, users, and decision-makers.
- Describe the desired business outcome.
- Capture current pain points and constraints.

Deliverable: problem statement, stakeholder list, and measurable goals.

02 — Define requirements

Suggested timing: Week 1–2

Turn ideas into testable requirements. Separate essential features from later enhancements.

- List functional requirements and user stories.
- Record non-functional needs such as security, accessibility, reliability, and performance.
- Define acceptance criteria for each important feature.
- Prioritise work as must-have, should-have, and later.

Deliverable: prioritised backlog and agreed acceptance criteria.

03 — Design the solution

Suggested timing: Week 2–3

Plan how the product should work before committing to implementation details.

- Sketch key user journeys and wireframes.
- Identify core data entities and relationships.
- Define API contracts and integration points.
- Choose a technology stack based on requirements and team capability.
- Review access control, privacy, backup, and failure scenarios.

Deliverable: reviewed designs, data model, and technical outline.

04 — Build in iterations

Suggested timing: based on scope

Deliver small, reviewable increments. Keep code versioned and make progress visible.

- Break work into tasks small enough to estimate and review.
- Set up repositories, environments, and a repeatable development workflow.
- Use code reviews and consistent naming/style conventions.
- Test changes as they are built; track defects and decisions.
- Demonstrate progress and re-prioritise when evidence changes.

Deliverable: tested increments and an updated backlog.

05 — Test and launch

Suggested timing: before release

Confirm the product meets its requirements and the team can operate it after launch.

- Test critical user journeys and error cases.
- Check permissions, configuration, logging, and data handling.
- Verify backups and recovery procedures.
- Prepare deployment, rollback, and support plans.
- Collect feedback and schedule post-launch improvements.

Deliverable: release checklist, deployment plan, and support ownership.

Project kickoff worksheet

Fill this in with the people who own the project outcome.

Project / product name	
Business problem to solve	
Primary users	
Desired outcome and how it will be measured	
Must-have features	
Out of scope for this release	
Key constraints / dependencies	
Decision-maker and project owner	
Target milestone or launch window	
Risks and assumptions	

Go-live readiness checklist

- ■ Requirements and acceptance criteria have been reviewed.
- ■ Critical user journeys and failure scenarios have been tested.
- ■ Access permissions and production configuration have been checked.
- ■ Monitoring, error reporting, and support ownership are defined.
- ■ Backup, recovery, and rollback procedures are understood.
- ■ Stakeholders know what is changing and where to report issues.
- ■ Post-launch review and improvement work are scheduled.

Remember: timelines depend on scope, team size, dependencies, and feedback. This guide is a planning framework, not a fixed delivery promise.

Prepared by Riyadvi Technologies • Use, adapt, and revisit this guide throughout the project.